1.Vaccum

```
# Simple Reflex Vacuum Cleaner Agent

def vacuum_agent(location, status):
    if status == "Dirty":
        return "Suck"
    elif location == "A":
        return "Move Right"
    else:
        return "Move Left"



# Initial environment
environment = {
    "A": "Dirty",
    "B": "Dirty"
}

location = "A"

print("Initial Environment:", environment)

# Agent actions
for i in range(4):

    status = environment[location]

    action = vacuum_agent(location, status)

    print("Location:", location)
    print("Status:", status)
    print("Action:", action)

    if action == "Suck":
        environment[location] = "Clean"
```

```
    elif action == "Move Right":
        location = "B"

    elif action == "Move Left":
        location = "A"

print("Final Environment:", environment)
```

3. A* Search

```
import heapq

goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)

# Calculate Manhattan Distance
def heuristic(state):
    h = 0

    for i, n in enumerate(state):
        if n != 0:
            j = goal.index(n)
            h += abs(i//3 - j//3) + abs(i%3 - j%3)

    return h

# A* Search Algorithm
def astar(start):
    pq = [(heuristic(start), 0, start)]
    parent = {start: None}
    cost = {start: 0}

    while pq:
        f, g, state = heapq.heappop(pq)
```

```
if g != cost[state]:
    continue

if state == goal:
    path = []

    while state is not None:
        path.append(state)
        state = parent[state]

    return path[::-1]

z = state.index(0)
r, c = z//3, z%3

moves = [(0,1), (0,-1), (1,0), (-1,0)]

for dr, dc in moves:
    nr, nc = r+dr, c+dc

    if 0 <= nr < 3 and 0 <= nc < 3:
        nz = nr*3 + nc

        new = list(state)
        new[z], new[nz] = new[nz], new[z]
        new = tuple(new)

        new_cost = g + 1

        if new_cost < cost.get(new, float('inf')):
            cost[new] = new_cost
            parent[new] = state

            heapq.heappush(
                pq, (new_cost + heuristic(new),
```

```
                    new_cost, new)
                )

    return None

# Initial State
start = (1, 2, 3, 4, 0, 6, 7, 5, 8)

result = astar(start)

if result:
    for state in result:
        print(state[:3])
        print(state[3:6])
        print(state[6:])
        print()
else:
    print("No solution")
```

5.TOWER OF HANOI

```
from collections import deque

def hanoi(n):
    start = (0,) * n
    goal = (2,) * n

    queue = deque([start])
    visited = {start: None}

    while queue:
        state = queue.popleft()

        if state == goal:
            path = []
```

```
            while state is not None:
                path.append(state)
                state = visited[state]

            return path[::-1]

        for disk in range(n):
            source = state[disk]

            # Check if disk is on top
            if any(state[i] == source for i in range(disk)):
                continue

            for dest in range(3):
                if dest == source:
                    continue

                # Check legal movement
                if any(state[i] == dest for i in range(disk)):
                    continue

                new = list(state)
                new[disk] = dest
                new = tuple(new)

                if new not in visited:
                    visited[new] = state
                    queue.append(new)

    return []

# Solve for 3 disks
path = hanoi(3)
```

```
for state in path:
    print(state)
```

6. SUDOKU

```
board = [
    [5,3,0,0,7,0,0,0,0],
    [6,0,0,1,9,5,0,0,0],
    [0,9,8,0,0,0,0,6,0],
    [8,0,0,0,6,0,0,0,3],
    [4,0,0,8,0,3,0,0,1],
    [7,0,0,0,2,0,0,0,6],
    [0,6,0,0,0,0,2,8,0],
    [0,0,0,4,1,9,0,0,5],
    [0,0,0,0,8,0,0,7,9]
]


def candidates(r, c):
    used = set(board[r])
    used.update(board[i][c] for i in range(9))

    for i in range(3):
        for j in range(3):
            used.add(board[r//3*3+i][c//3*3+j])

    return set(range(1, 10)) - used


def solve():
    best = None
    options = None

    for r in range(9):
        for c in range(9):
            if board[r][c] == 0:
                values = candidates(r, c)

                if not values:
```

```
                return False

            if options is None or len(values) < len(options):
                best = (r, c)
                options = values

    if best is None:
        return True

    r, c = best

    for n in sorted(options):
        board[r][c] = n

        if solve():
            return True

        board[r][c] = 0

    return False

if solve():
    for row in board:
        print(row)
else:
    print("No solution")
```

B2. DFS

```
def dfs():
    stack = [(0, 0)]
    visited = set()

    while stack:
        x, y = stack.pop()

        if (x, y) in visited:
```

```
            continue

        visited.add((x, y))
        print("State:", (x, y))

        if x == 2 or y == 2:
            print("Goal Reached!")
            return

        states = [
            (4, y), (x, 3), (0, y), (x, 0),
            (x-min(x, 3-y), y+min(x, 3-y)),
            (x+min(y, 4-x), y-min(y, 4-x))
        ]

        for s in states:
            if s not in visited:
                stack.append(s)

    print("No solution")

dfs()
```

B3.QUEENS

```
N = 8
board = [-1] * N

def safe(row, col):
    for i in range(row):
        if board[i] == col:
            return False

        if abs(board[i] - col) == abs(i - row):
            return False

    return True
```

```
def solve(row):
    if row == N:
        return True

    for col in range(N):
        if safe(row, col):
            board[row] = col

            if solve(row + 1):
                return True

            board[row] = -1

    return False

if solve(0):
    for i in range(N):
        print(" ".join(
            "Q" if board[i] == j else "."
            for j in range(N)
        ))
else:
    print("No solution")
```

B4.CHATBOT

```
def chatbot(user):
    user = user.lower()

    if "hello" in user or "hi" in user:
        return "Hello! How can I help you?"

    elif "name" in user:
        return "I am an AI chatbot."

    elif "how are you" in user:
```

```
        return "I am fine. Thank you!"

    elif "course" in user:
        return "I can help you with AI concepts."

    elif "bye" in user:
        return "Goodbye!"

    else:
        return "Sorry, I don't understand."

print("Chatbot: Hello! Type bye to exit.")

while True:
    user = input("You: ")
    print("Chatbot:", chatbot(user))

    if "bye" in user.lower():
        break
```

B5.INTELLIGENT AGENT (HIGHER/LOWER)

```
low = 1
high = 100

print("Think of a number between 1 and 100.")

while low <= high:
    guess = (low + high) // 2

    print("My guess is:", guess)
    feedback = input("Enter higher, lower or correct: ").lower()

    if feedback == "higher":
        low = guess + 1

    elif feedback == "lower":
```

```
        high = guess - 1

    elif feedback == "correct":
        print("Number guessed successfully!")
        break

    else:
        print("Invalid feedback!")

else:
    print("Inconsistent feedback!")
```

A.2 a model-based intelligent agent

```
from collections import deque

grid = [
    [0, 0, 0, 0, 0],
    [0, 1, 1, 0, 0],
    [0, 0, 0, 0, 1],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 0, 0]
]

start = (0, 0)
goal = (4, 4)

# Internal state of the agent
position = start
visited = set()

def bfs(start):
    queue = deque([[start]])
    seen = {start}

    while queue:
        path = queue.popleft()
```

```
        r, c = path[-1]

        if (r, c) == goal:
            return path

        for dr, dc in [(0,1), (0,-1), (1,0), (-1,0)]:
            nr, nc = r + dr, c + dc

            if (0 <= nr < 5 and 0 <= nc < 5
                and grid[nr][nc] == 0
                and (nr, nc) not in seen):

                seen.add((nr, nc))
                queue.append(path + [(nr, nc)])

    return None

path = bfs(position)

if path:
    for position in path:
        visited.add(position)
        print("Agent Position:", position)

    print("Goal Reached!")
else:
    print("No Path Found!")
```

A4. ALPHA BETA PRUNING

```
def minimax(depth, node, maximizing, alpha, beta, values):

    if depth == 3:
        return values[node]

    if maximizing:
        best = float('-inf')
```

```
        for i in range(2):
            val = minimax(depth+1, node*2+i,
                          False, alpha, beta, values)

            best = max(best, val)
            alpha = max(alpha, best)

            if beta <= alpha:
                break

        return best

    else:
        best = float('inf')

        for i in range(2):
            val = minimax(depth+1, node*2+i,
                          True, alpha, beta, values)

            best = min(best, val)
            beta = min(beta, best)

            if beta <= alpha:
                break

        return best


values = [3, 5, 6, 9, 1, 2, 0, -1]

result = minimax(0, 0, True,
                 float('-inf'), float('inf'), values)

print("Optimal value:", result)
```

A7. TRAVELLING SALESMAN

```
import random


cities = 5

k = 2


distance = [

   [0, 2, 9, 10, 7],

   [2, 0, 6, 4, 3],

   [9, 6, 0, 8, 5],

   [10, 4, 8, 0, 6],

   [7, 3, 5, 6, 0]

]


def cost(route):

   total = 0

   for i in range(cities):

      total += distance[route[i]][route[(i+1) % cities]]

   return total


# Initial beam

beam = [random.sample(range(cities), cities) for _ in range(k)]


for _ in range(100):

   new_beam = []


   for route in beam:

      for i in range(cities):

         for j in range(i+1, cities):

            new = route[:]

            new[i], new[j] = new[j], new[i]

            new_beam.append(new)


   beam = sorted(new_beam, key=cost)[:k]
```

```
best = min(beam, key=cost)

print("Best Route:", best + [best[0]])
print("Minimum Distance:", cost(best))
```

B1.BFS

```
from collections import deque

grid = [
    [0, 0, 0, 0],
    [0, 1, 1, 0],
    [0, 0, 0, 0],
    [1, 1, 0, 0]
]

start = (0, 0)
goal = (3, 3)

def bfs():
    queue = deque([[start]])
    visited = {start}

    while queue:
        path = queue.popleft()
        r, c = path[-1]

        if (r, c) == goal:
            return path

        for dr, dc in [(0,1), (1,0), (0,-1), (-1,0)]:
            nr, nc = r + dr, c + dc

            if (0 <= nr < 4 and 0 <= nc < 4
                and grid[nr][nc] == 0
                and (nr, nc) not in visited):
```

```
            visited.add((nr, nc))
            queue.append(path + [(nr, nc)])

    return None

path = bfs()

if path:
    print("Path found:")
    for position in path:
        print(position)
else:
    print("No path found")
```

B6. GOAL STACK PLANNING

```
initial = {'A': 'Table', 'B': 'A', 'C': 'Table'}
goal = {'A': 'Table', 'B': 'Table', 'C': 'B'}

print("Initial State:", initial)

# Goal stack containing actions in reverse order
stack = [
    "Stack C on B",
    "Pick up C",
    "Put B on Table",
    "Unstack B from A"
]

while stack:
    action = stack.pop()
    print("Action:", action)

    if action == "Unstack B from A":
        initial['B'] = 'Table'
```

```
    elif action == "Put B on Table":
        initial['B'] = 'Table'

    elif action == "Stack C on B":
        initial['C'] = 'B'

print("Final State:", initial)
print("Goal Achieved:", initial == goal)
```

B7. GAME PLAYING

```
def minimax(board, is_max):
    if "XXX" in lines(board):
        return 1
    if "OOO" in lines(board):
        return -1
    if " " not in board:
        return 0

    if is_max:
        best = -999
        for i in range(9):
            if board[i] == " ":
                board[i] = "X"
                best = max(best, minimax(board, False))
                board[i] = " "
        return best

    else:
        best = 999
        for i in range(9):
            if board[i] == " ":
                board[i] = "O"
                best = min(best, minimax(board, True))
                board[i] = " "
        return best
```

```
def lines(b):
    return [
        b[0]+b[1]+b[2], b[3]+b[4]+b[5],
        b[6]+b[7]+b[8], b[0]+b[3]+b[6],
        b[1]+b[4]+b[7], b[2]+b[5]+b[8],
        b[0]+b[4]+b[8], b[2]+b[4]+b[6]
    ]



def computer_move(board):
    best = -999
    move = -1

    for i in range(9):
        if board[i] == " ":
            board[i] = "X"
            score = minimax(board, False)
            board[i] = " "

            if score > best:
                best = score
                move = i

    return move



board = [" "] * 9

while True:
    move = computer_move(board)
    board[move] = "X"

    print("\n", board[0:3])
    print(board[3:6])
```

```
    print(board[6:9])

    if "XXX" in lines(board):
        print("Computer Wins!")
        break

    if " " not in board:
        print("Draw!")
        break

    user = int(input("Enter position (0-8): "))

    if user < 0 or user > 8 or board[user] != " ":
        print("Invalid position!")
        continue

    board[user] = "O"

    if "OOO" in lines(board):
        print("You Win!")
        break
```

C1 MEDICAL DIAGNOSIS

```
rules = {
    "Flu": ["fever", "cough", "body_pain"],
    "Cold": ["sneezing", "runny_nose", "cough"],
    "Allergy": ["sneezing", "itchy_eyes"],
    "Malaria": ["fever", "chills", "sweating"]
}

facts = {"fever", "cough", "body_pain"}

# Forward Chaining
print("Forward Chaining:")
```

```
for disease, symptoms in rules.items():
    if all(s in facts for s in symptoms):
        facts.add(disease)
        print("Possible condition:", disease)

# Backward Chaining
def backward(goal):
    if goal in facts:
        return True

    if goal not in rules:
        return False

    return all(backward(s) for s in rules[goal])

print("\nBackward Chaining:")

goal = "Flu"

if backward(goal):
    print("Goal proved:", goal)
else:
    print("Goal not proved:", goal)
```